

If both projects share the same Python installation, you might face compatibility issues—the new libraries might conflict with the older ones. This is where virtual environments come in. They're like separate workshops for each of your projects, ensuring that the tools for one project don't interfere with another.

Why Virtual Environments Are Critical for Trading

In 2018, a London-based hedge fund lost approximately \$300,000 in a single day when a recently updated machine learning library produced unexpected results in their production trading system. The issue? The development team had tested the new library with their strategy, but didn't realize it would affect another algorithm running in the same environment. Virtual environments would have prevented this costly error.

Creating Virtual Environments

Python comes with a built-in tool called `venv` for creating virtual environments. Here's how to use it:

1.

Create a new environment: In your terminal or command prompt, navigate to your project directory and run:

```
1. python -m venv myenv
```

2.

This creates a new virtual environment named "myenv" in your current directory.

3.

Activate the environment: The activation command differs by operating system:

```
1. # On Windows:  
2. myenv\\Scripts\\activate  
3.  
4. # On macOS and Linux:  
5. source myenv/bin/activate  
6.
```